K-Means Clustering: Cost Function, Convergence & Worked Examples

Basics of Data Mining — Week 8

Last time we learnt the *recipe* of k-means: pick centroids, assign points, update centroids, repeat. This lecture asks the more interesting question: **why does that recipe work?** We will (1) prove that the **mean** is not an arbitrary choice for the centroid, (2) discuss the knobs we must set ourselves (number of clusters, initialization, scaling), (3) trace two complete examples by hand so that you can code k-means from scratch, and (4) see why the algorithm is **guaranteed to converge**.

Index

1. Recap: The Cost Function — Meet J, the grumpy accountant who adds up every point's complaint about its centroid.
2. Why the Mean? — Calculus crashes the party and proves the mean is no accident.
3. Mean vs. Median vs. the Rest — Outliers, drama, and why "robust" is not the same as "justified".
4. How Many Clusters? — The question with no official answer (only educated guesses).
5. Example 1: Six Numbers, Two Clusters — The warm-up lap: one dimension, zero surprises.
6. Example 2: Spenders and Savers in 2D — Real arithmetic, real fractions, no skipped steps.
7. Why Does K-Means Converge? — Two steps, both going downhill, so the ball must stop rolling.
8. The Three Practical Knobs — Initialization, number of clusters, scaling: the things the algorithm will not decide for you.
9. K-Means as an Optimization Problem — Stop memorising the recipe, start seeing the objective.
10. Practice Questions — Five problems that will make you think twice (at least).

Recap: The Cost Function

K-means clustering is driven by one quantity, the **cost function** (also called the objective or distortion):

$$J = \sum_{k=1}^{c} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Every symbol has a job:

Symbol	Meaning
c	Number of clusters (also written C or K)
C_k	The set of points currently assigned to the k-th cluster
x_i	The i-th data point (a feature vector)
μ_k	The **mean vector** (centroid) of cluster k: the mean of the features of all points in that cluster
$\|x_i - \mu_k\|^2$	Squared **Euclidean distance** between a point and its cluster centroid

The two summations say: *go over every cluster, and inside each cluster go over every point that belongs to it*. J is therefore the **total squared distance of all points from their own centroids**. Small J means tight clusters.

By definition, the centroid is

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

This raises the question that drives the first half of the lecture: **is the mean special here?** The data x_i is given, μ_k is "the centre of cluster k", and we *chose* to make it the mean. Could we have used the median, or something else?

Why the Mean?

Setting. Take n observations $x_1, x_2, \ldots, x_n$. Pick *any* candidate centre μ (we do not yet assume it is the mean) and define

$$J(\mu) = \sum_{i=1}^{n} (x_i - \mu)^2$$

Question: for which value of μ is $J(\mu)$ minimum?

Step 1: Expand the square

Using $(a-b)^2 = a^2 - 2ab + b^2$:

$$J(\mu) = \sum_{i=1}^{n} (x_i^2 - 2x_i\mu + \mu^2)$$

Step 2: Differentiate with respect to μ

We want to minimize J with respect to μ, so μ is the variable and each x_i is a constant. The term x_i^2 does not depend on μ, so its derivative is 0:

$$\frac{dJ(\mu)}{d\mu} = \sum_{i=1}^{n} (-2x_i + 2\mu)$$

Step 3: Set the derivative to zero

Basic calculus: a minimum occurs where the slope vanishes.

$$\frac{dJ(\mu)}{d\mu} = 0 \implies \sum_{i=1}^{n} (-2x_i + 2\mu) = 0$$

Split the summation. The μ term is added n times, so it becomes $2n\mu$:

$$-2\sum_{i=1}^{n} x_i + 2n\mu = 0$$

Step 4: Solve for μ

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

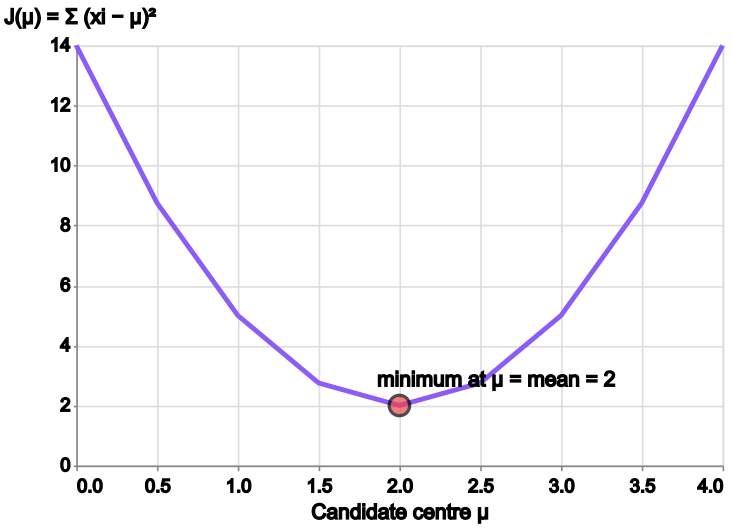
This is exactly the **arithmetic mean**. We found nothing new about the mean itself, but we found something new about *why we use it*: the mean is the point that **minimizes the sum of squared distances** to the data. That is the reason it appears in k-means, and it is not an arbitrary choice.

Step 5: Confirm it is a minimum, not a maximum

$$\frac{d^2J}{d\mu^2} = 2n > 0$$

The second derivative is positive, so the curve is a upward-opening parabola and the stationary point is a true minimum.

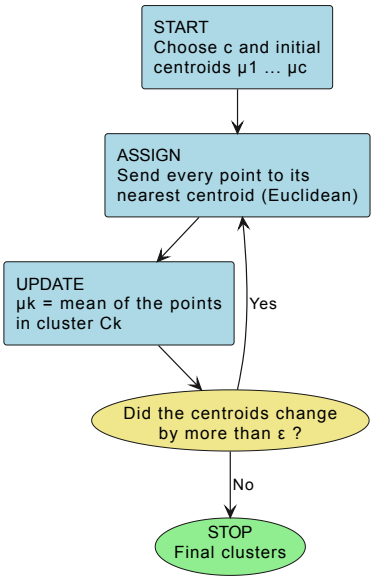
For the data $\{1, 2, 3\}$ the cost is $J(\mu) = (1-\mu)^2 + (2-\mu)^2 + (3-\mu)^2 = 3\mu^2 - 12\mu + 14$. At $\mu = 2$ it equals $1 + 0 + 1 = 2$, and moving μ in either direction only makes it worse.



> **Note for Review** Pro-Tip: This often appears on exams regarding *why k-means uses the mean*. Remember two facts:
>
> 1. **Vector data:** the derivation works one coordinate at a time, so the centroid of 2D points is the *component-wise* mean (mean of spending, mean of saving).
> 2. **Link to variance:** if you plug $\mu = \bar{x}$ back in, $J(\bar{x}) = \sum(x_i - \bar{x})^2 = n\sigma^2$. This is why, at the mean, the cost is just n times the variance.
>
> A neat algebraic proof without calculus: write $x_i - \mu = (x_i - \bar{x}) + (\bar{x} - \mu)$. The cross term vanishes because $\sum(x_i - \bar{x}) = 0$, leaving $J(\mu) = \sum_{i=1}^{n}(x_i - \bar{x})^2 + n(\bar{x} - \mu)^2$ The first part is fixed; the second part is
>
> ≥ 0 and is zero only when $\mu = \bar{x}$.

Story version. Three friends live at positions 1, 2 and 3 on a street, and you must place one coffee kiosk. If every friend's "unhappiness" is their *squared* walking distance, the happiest overall spot is the mean, 2. Each centroid in k-means is exactly such a kiosk for its cluster of "friends".

The algorithm at a glance

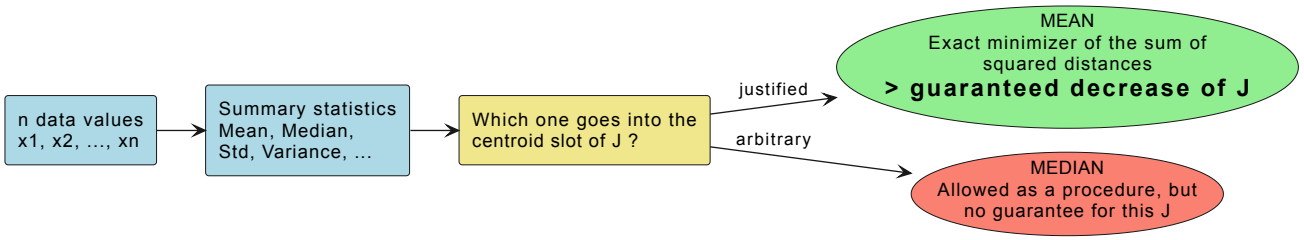


Mean vs. Median vs. the Rest

For n numbers $x_1, \ldots, x_n$ we have many **summary statistics** available: mean, median, mode, standard deviation, variance, skewness, kurtosis, box-plot quantities, and so on. In machine learning and AI we constantly face choices like "which one do I plug in here?". The rule to remember:

> **Choices in an algorithm must have mathematical backing; they must not look arbitrary.**

In k-means we *could* write the cost as $\sum_k \sum_{x_i \in C_k} \|x_i - m_k\|^2$ with m_k the **median** of cluster k. Procedurally nothing stops us. The problem is that the algorithm is an **iterative optimization** (centroids are updated again and again), so we need a *guarantee* that each update reduces the cost. For the squared-distance cost, only the mean carries that guarantee, as we just proved.



"But the mean is sensitive to outliers!"

The single value 100 drags the mean towards a place where none of the other three observations live, while the median barely moves. So the median is "more robust". However, this argument is **neither practical nor theoretical**:

- **Not practical:** we cannot actually *define* an outlier just by looking at data. In this toy example 100 is obviously far from $1, 2, 4$, but in physical systems a strange value may come from a faulty sensor or machine, and it is still part and parcel of the signal.
- **Not theoretical:** robustness is a property of the statistic, whereas k-means needs a statistic that **minimizes its own cost function**.

True, and it is a line you will often hear. Consider $\{1, 2, 4, 100\}$:

Data	Mean	Median
$\{1, 2, 4\}$	$\frac{1+2+4}{3} \approx 2.33$	2
$\{1, 2, 4, 100\}$	$\frac{1+2+4+100}{4} = \frac{107}{4} = 26.75$	$\frac{2+4}{2} = 3$

So k-means uses the mean for a *separate, mathematical* reason: it is the minimizer of the squared-distance cost.

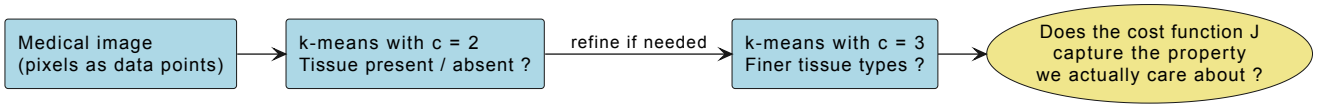
> **Note for Review** Pro-Tip: A common follow-up question: "what does the median minimize?" Answer: the sum of **absolute** deviations $\sum |x_i - m|$. Pairing the median with absolute distance gives a *different*, self-consistent algorithm (k-medians). Mixing the median with *squared* distance breaks the guarantee.

How Many Clusters?

The number of clusters c must be supplied by us, and again there is **no definitive answer** to "how do I choose it?". It is empirical:

- Start with $c = 2$ (the minimum meaningful value) and increase it, checking whether the result fits the situation and is useful.
- The right value depends on the **kind of data** and on your **eventual goal**.
- Sometimes domain knowledge makes it obvious: if you are looking for three specific properties in the data, start with $c = 3$ and hope the clustering recovers them.

Example: medical image segmentation. K-means is widely used for this. Suppose a scan may contain different kinds of tissue and you only want to know whether a certain tissue (for example, tumour tissue) is *present or absent*. Start with $c = 2$ and see how k-means performs; if a finer segmentation helps, try $c = 3$.



No guarantee. K-means has a perfectly well-defined cost function, but whether *that* cost function captures the property you want (a tissue type, a customer behaviour) may or may not hold. This is true of any data mining algorithm: it is a general, mathematically well-defined procedure, not something designed for one particular application. Deciding where it is more applicable is up to you, with no a priori answers.

> **Note for Review** Pro-Tip: Be careful with the tempting idea "choose the c that gives the smallest J". J **always decreases (or stays equal) as** c **grows**, and reaches 0 when $c = n$ (every point is its own cluster). So minimizing J alone cannot choose c. In practice people look for the point where the improvement in J flattens (the "elbow") *and* check it against domain knowledge.

Example 1: Six Numbers, Two Clusters

Data and goal.

$$D = \{1, 2, 3, 8, 9, 10\}, \qquad c = 2$$

By eye the answer is obvious (small numbers $1, 2, 3$ versus large numbers $8, 9, 10$). The point is to watch k-means reach it step by step. Since the data is one-dimensional, each centroid is a single number.

Step 1: Initial centroids

In theory the starting point does not matter; in practice it can influence where the optimization ends (as with gradient descent). We simply start with

$$\mu_1 = 2, \qquad \mu_2 = 9$$

Step 2: Iteration 1, assignment

Each point goes to the nearest centroid. In one dimension the Euclidean distance is the absolute difference (using squared differences would lead to the same decisions):

Data x	$\lvert x - \mu_1 \rvert$	$\lvert x - \mu_2 \rvert$	Cluster
1	$\lvert 1-2 \rvert = 1$	$\lvert 1-9 \rvert = 8$	C_1
2	$\lvert 2-2 \rvert = 0$	$\lvert 2-9 \rvert = 7$	C_1
3	$\lvert 3-2 \rvert = 1$	$\lvert 3-9 \rvert = 6$	C_1
8	$\lvert 8-2 \rvert = 6$	$\lvert 8-9 \rvert = 1$	C_2
9	$\lvert 9-2 \rvert = 7$	$\lvert 9-9 \rvert = 0$	C_2
10	$\lvert 10-2 \rvert = 8$	$\lvert 10-9 \rvert = 1$	C_2

So

$$C_1 = \{1, 2, 3\}, \qquad C_2 = \{8, 9, 10\}$$

The cost with these centroids is

$$J = \underbrace{(1-2)^2 + (2-2)^2 + (3-2)^2}_{=2} + \underbrace{(8-9)^2 + (9-9)^2 + (10-9)^2}_{=2} =$$

Step 3: Update the centroids

Each new centroid is the mean of the points in its cluster:

$$\mu_1 = \frac{1+2+3}{3} = \frac{6}{3} = 2, \qquad \mu_2 = \frac{8+9+10}{3} = \frac{27}{3} = 9$$

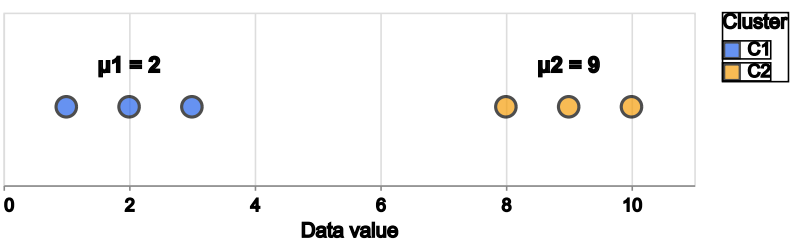
The centroids are exactly the same as before. This particular example is so simple (and we started so luckily) that nothing moves.

Step 4: Stopping criterion

Stop when the centroids **do not change**. In practice we compare consecutive iterations k and $k+1$ and stop when the update is tiny:

$$|\mu_k - \mu_{k+1}| < \epsilon, \qquad \text{for example } \epsilon = 10^{-3}$$

Here the change is exactly 0, so we stop. (One may equally monitor the cost J and stop when it no longer decreases.)



Example 2: Spenders and Savers in 2D

Now the same procedure in two dimensions. Six customers are described by (spending, saving), in arbitrary scaled units:

Customer	(Spending, Saving)
A	$(1, 1)$
B	$(2, 1)$
C	$(2, 2)$
D	$(8, 8)$
E	$(9, 8)$
F	$(8, 9)$

We want $c = 2$ clusters. Each centroid now has **two coordinates**, because the data is 2D. For 2D points (a_1, a_2) and (b_1, b_2) we use the squared Euclidean distance

$$d = (a_1 - b_1)^2 + (a_2 - b_2)^2$$

(Comparing squared distances picks the same nearest centroid as comparing distances, since the square root is monotonic, and squared distance is exactly what J uses.)

Step 1: Initial centroids

$$\mu_1 = (1, 1), \qquad \mu_2 = (9, 8)$$

Step 2: Iteration 1, assignment, customer by customer

Customer A $(1, 1)$

$$d(A, \mu_1) = (1 - 1)^2 + (1 - 1)^2 = 0$$
$$d(A, \mu_2) = (1 - 9)^2 + (1 - 8)^2 = 64 + 49 = 113$$

$0 < 113$, so $A \in C_1$.

Customer B $(2, 1)$

$$d(B, \mu_1) = (2 - 1)^2 + (1 - 1)^2 = 1$$
$$d(B, \mu_2) = (2 - 9)^2 + (1 - 8)^2 = 49 + 49 = 98$$

$1 < 98$, so $B \in C_1$.

Customer C $(2, 2)$

$$d(C, \mu_1) = (2 - 1)^2 + (2 - 1)^2 = 2$$
$$d(C, \mu_2) = (2 - 9)^2 + (2 - 8)^2 = 49 + 36 = 85$$

$2 < 85$, so $C \in C_1$.

Customer D $(8, 8)$

$$d(D, \mu_1) = (8 - 1)^2 + (8 - 1)^2 = 49 + 49 = 98$$
$$d(D, \mu_2) = (8 - 9)^2 + (8 - 8)^2 = 1$$

$1 < 98$, so $D \in C_2$.

Customer E $(9, 8)$

$$d(E, \mu_1) = (9 - 1)^2 + (8 - 1)^2 = 64 + 49 = 113, \qquad d(E, \mu_2) = 0$$

So $E \in C_2$.

Customer F $(8, 9)$

$$d(F, \mu_1) = (8 - 1)^2 + (9 - 1)^2 = 49 + 64 = 113, \qquad d(F, \mu_2) = (8 - 9)^2 + (9 - 8)^2 = 2$$

So $F \in C_2$.

Summary of iteration 1

Customer	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	Cluster
A	0	113	C_1
B	1	98	C_1
C	2	85	C_1
D	98	1	C_2
E	113	0	C_2
F	113	2	C_2

$$C_1 = \{A, B, C\}, \qquad C_2 = \{D, E, F\}$$

Cost after iteration 1 (each point's distance to its own centroid):

$$J = (0 + 1 + 2) + (1 + 0 + 2) = 6$$

Step 3: Iteration 2, update the centroids

Take the mean of the points of each cluster, **separately along each dimension** (spending, then saving).

New μ_1 from $A(1,1),\ B(2,1),\ C(2,2)$:

$$\mu_1^{\text{new}} = \left(\frac{1+2+2}{3},\ \frac{1+1+2}{3}\right) = \left(\frac{5}{3}, \frac{4}{3}\right) \approx (1.67,\ 1.33)$$

New μ_2 from $D(8,8),\ E(9,8),\ F(8,9)$:

$$\mu_2^{\text{new}} = \left(\frac{8+9+8}{3},\ \frac{8+8+9}{3}\right) = \left(\frac{25}{3}, \frac{25}{3}\right) \approx (8.33,\ 8.33)$$

Step 4: Iteration 2, re-assignment with the new centroids

Customer B $(2,1)$

$$d(B,\mu_1^{\text{new}}) = (2-\tfrac{5}{3})^2 + (1-\tfrac{4}{3})^2 = \tfrac{1}{9} + \tfrac{1}{9} = \tfrac{2}{9} \approx 0.22$$

$$d(B,\mu_2^{\text{new}}) = (2-\tfrac{25}{3})^2 + (1-\tfrac{25}{3})^2 = \tfrac{361}{9} + \tfrac{484}{9} = \tfrac{845}{9} \approx 93.9$$

So B stays in C_1.

Customer D $(8,8)$

$$d(D,\mu_1^{\text{new}}) = (8-\tfrac{5}{3})^2 + (8-\tfrac{4}{3})^2 = \tfrac{361}{9} + \tfrac{400}{9} = \tfrac{761}{9} \approx 84.6$$

$$d(D,\mu_2^{\text{new}}) = (8-\tfrac{25}{3})^2 + (8-\tfrac{25}{3})^2 = \tfrac{1}{9} + \tfrac{1}{9} = \tfrac{2}{9} \approx 0.22$$

So D stays in C_2. The remaining customers are computed the same way:

Customer	$d(\cdot,\mu_1^{\text{new}})$	$d(\cdot,\mu_2^{\text{new}})$	Cluster
A	$\frac{5}{9} \approx 0.56$	$\frac{968}{9} \approx 107.56$	C_1
B	$\frac{2}{9} \approx 0.22$	$\frac{845}{9} \approx 93.89$	C_1
C	$\frac{5}{9} \approx 0.56$	$\frac{722}{9} \approx 80.22$	C_1
D	$\frac{761}{9} \approx 84.56$	$\frac{2}{9} \approx 0.22$	C_2
E	$\frac{884}{9} \approx 98.22$	$\frac{5}{9} \approx 0.56$	C_2
F	$\frac{890}{9} \approx 98.89$	$\frac{5}{9} \approx 0.56$	C_2

No customer changed cluster, so $C_1 = \{A,B,C\}$ and $C_2 = \{D,E,F\}$ again. The cost has dropped:

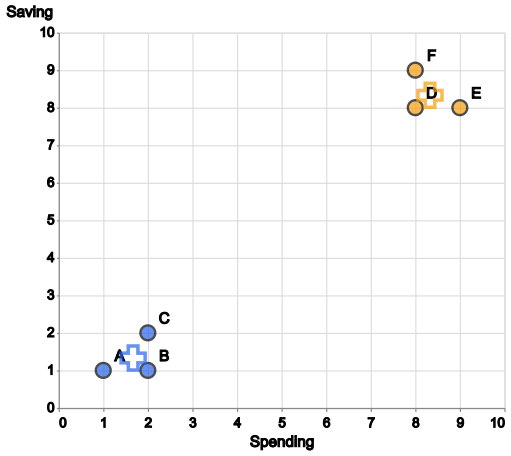
$$J = \frac{5+2+5}{9} + \frac{2+5+5}{9} = \frac{12}{9} + \frac{12}{9} = \frac{24}{9} \approx 2.67$$

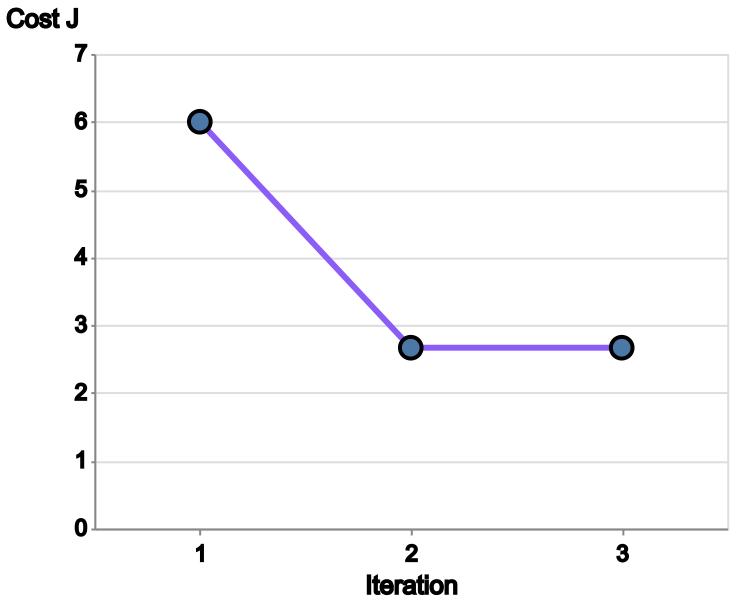
Step 5: Check the stopping criterion

Recomputing the means of the unchanged clusters gives $\mu_1 = (\frac{5}{3}, \frac{4}{3})$ and $\mu_2 = (\frac{25}{3}, \frac{25}{3})$ again, so the centroids no longer change and the algorithm **stops**.

The cost only goes down

Stage	Centroids used	J
Iteration 1	$(1,1)$ and $(9,8)$	6
Iteration 2	$(1.67, 1.33)$ and $(8.33, 8.33)$	2.67
Iteration 3 (check)	unchanged	2.67





Why Does K-Means Converge?

Remember the cost function:

$$J = \sum_{k=1}^{c} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

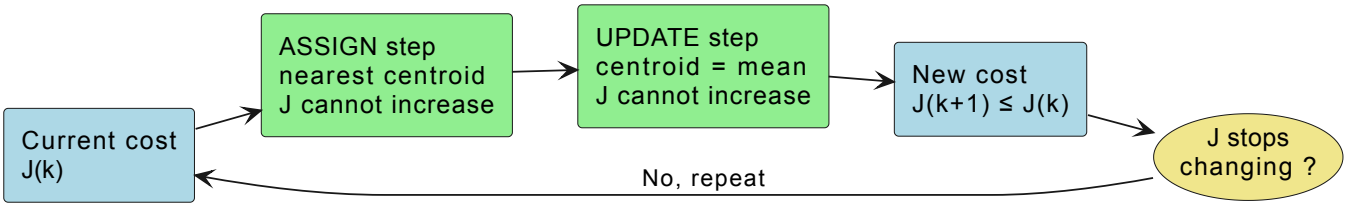
Each iteration of k-means does two things, and **each one can only lower J (or leave it unchanged)**:

1. **Assignment step (centroids fixed).** Every point moves to its *nearest* centroid. A point's contribution to J is its squared distance to its centroid, so choosing the nearest one makes that contribution as small as possible. Therefore J cannot increase.
2. **Update step (assignments fixed).** Each centroid is set to the *mean* of its cluster. We proved above that the mean minimizes the sum of squared distances, so J cannot increase here either.

Chaining the two steps gives the guarantee:

$$J^{(k+1)} \leq J^{(k)}$$

The cost at iteration $k+1$ is never larger than at iteration k. This is exactly why using the **mean** matters: if the update step used something else, the second half of the argument would collapse.



Since $J \geq 0$ and the cost never goes up, it must level out: either it keeps decreasing, or it stays almost the same, in which case we stop. This is the same idea as **gradient descent**:

	Gradient descent	K-means
What we minimize	Loss function	Cost J
Each step	Move down the slope	Assign, then update centroids
Cost over iterations	Decreases	Decreases (or stays equal)
When we stop	Gradient is 0, nothing left to update	Centroids (or J) stop changing

You are "climbing down the hill one step at a time" and eventually reach a minimum where the next step does not help.

> **Note for Review** Pro-Tip: This often appears on exams regarding *convergence*. Remember that the guarantee is a **theoretical** one for the cost function, with the mean as the update. It promises convergence to a *minimum of J reachable from the chosen start*, which is typically a local minimum. This is why initialization matters in practice, and why small practical differences can occur (but are usually tackled easily).

The Three Practical Knobs

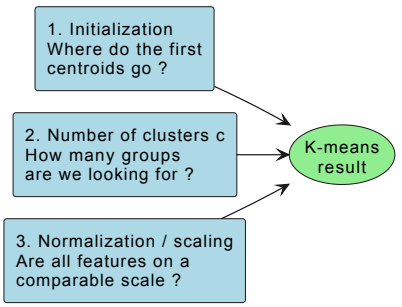
1. Initialization

In theory you can start anywhere; in practice the starting point can affect the final result. A common practical choice is to take **actual data points** as the initial centroids (one data point per cluster).

2. Number of clusters c

Covered above: it is a parameter that the application (with domain knowledge) must define; it cannot be arbitrary.

K-means is easy to compute (a distance and a mean), but three things are *not* decided by the algorithm:



3. Normalization or scaling

K-means computes distances, so a feature with huge magnitude can dominate the distance. Imagine the data (income, number of monthly transactions), for example $(500000,\ 92)$. Income is of order 10^5 while transactions are of order 10^2; in a squared distance the income term can overwhelm the other (it is not *guaranteed* to dominate, but it easily can). Hence some **feature scaling** is typically applied *before* k-means. A standard choice is **standardization** (z-score):

$$z = \frac{x - \mu_x}{\sigma_x}$$

Here μ_x and σ_x are the mean and standard deviation of that feature (column). Other normalizations are also fine.

Worked example. Four customers described by (income, a score between 0 and 1):

Customer	Income	Score
P1	5000	0.1
P2	6274	0.9
P3	4947	0.6
P4	3888	0.92

Income column.

$$\mu = \frac{5000 + 6274 + 4947 + 3888}{4} = \frac{20109}{4} = 5027.25$$

Deviations: $-27.25,\ 1246.75,\ -80.25,\ -1139.25$. Their squares sum to $2{,}859{,}458.75$, so

$$\sigma = \sqrt{\frac{2{,}859{,}458.75}{4}} \approx 845.5$$

Score column.

$$\mu = \frac{0.1 + 0.9 + 0.6 + 0.92}{4} = \frac{2.52}{4} = 0.63, \qquad \sigma = \sqrt{\frac{0.4388}{4}} \approx 0.331$$

Standardized data ($z = (x - \mu)/\sigma$):

Customer	z_{income}	z_{score}
P1	$\frac{-27.25}{845.5} \approx -0.03$	$\frac{-0.53}{0.331} \approx -1.60$
P2	$\frac{1246.75}{845.5} \approx 1.47$	$\frac{0.27}{0.331} \approx 0.82$
P3	$\frac{-80.25}{845.5} \approx -0.09$	$\frac{-0.03}{0.331} \approx -0.09$
P4	$\frac{-1139.25}{845.5} \approx -1.35$	$\frac{0.29}{0.331} \approx 0.88$

The effect on one distance (P1 to P2).

Before scaling:

$$d = (6274 - 5000)^2 + (0.9 - 0.1)^2 = 1274^2 + 0.8^2 = 1{,}623{,}076 + 0.64$$

The score feature contributes essentially nothing. After scaling:

$$d = (1.47 - (-0.03))^2 + (0.82 - (-1.60))^2 \approx 1.51^2 + 2.42^2 \approx 2.27 + 5.84 \approx 8.11$$

Now **both features matter**. This is why normalization before k-means helps avoid problems arising from feature mismatch.

K-Means as an Optimization Problem

If you remember only one idea from this topic, make it this:

> **Think of k-means as an optimization problem, not as a procedure.**

The procedure (initialize, compute distances to each centroid, assign, update centroids, repeat) is easy and not conceptually deep. If you stop there, you will not understand *why* it was designed that way. The real content is:

- **Objective:** minimize the Euclidean cost $J = \sum_{k=1}^{c} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$ over all clusters.
- **Why the mean:** it is the exact minimizer of the squared-distance cost.
- **Why it converges:** both steps lower J, step after step.

K-means is also used in advanced applications. For instance, in **medical images**, segmenting tumour cells is a task where k-means is typically the *first step* (other clustering methods exist, each with advantages and disadvantages). Further interesting aspects, such as k-means being linear versus non-linear, are beyond this course.

Implementing it from scratch

Everything above maps directly onto a few lines of NumPy (no clustering package needed). `X` is an $n \times d$ array and `mu` the initial $c \times d$ centroids:

Try it on Example 2 with `X = np.array([[1,1],[2,1],[2,2],[8,8],[9,8],[8,9]])` and `mu = np.array([[1,1],[9,8]], dtype=float)`; you should reproduce the centroids (1.67, 1.33) and (8.33, 8.33). (Edge case worth thinking about: what happens if a cluster ends up empty?)

```
import numpy as np

def kmeans(X, mu, eps=1e-3, max_iter=100):
    for _ in range(max_iter):
        # ASSIGN: squared distance of every point to every centroid
        d = ((X[:, None, :] - mu[None, :, :]) ** 2).sum(axis=2)
        labels = d.argmin(axis=1)
        # UPDATE: each centroid becomes the mean of its cluster
        new_mu = np.array([X[labels == k].mean(axis=0)
                           for k in range(len(mu))])
        # STOP: centroids barely moved
        if np.abs(new_mu - mu).max() < eps:
            break
        mu = new_mu
    J = d[np.arange(len(X)), labels].sum()
    return mu, labels, J
```

Practice Questions

1. **A bad start.** Take the data $\{1, 2, 3, 8, 9, 10\}$ with $c = 2$ and deliberately start with both centroids in the left group, $\mu_1 = 1$ and $\mu_2 = 2$. Run k-means by hand, writing out every assignment and every mean. Record J after each iteration. Does the algorithm recover the "obvious" clusters? What does this tell you about the claim that, *in theory*, the starting point does not matter?

2. **The hijacked cluster.** Add one extra observation, 100, to the same data (so $D = \{1, 2, 3, 8, 9, 10, 100\}$) and start from $\mu_1 = 2$, $\mu_2 = 9$. Follow the iterations until the centroids stop moving. Describe what the final clusters look like and explain, using the derivation of why the mean minimizes J, which part of the algorithm lets a single point cause this. Would the median have rescued you, and what would you have to change in the *cost function* for that rescue to be legitimate?

3. **When scaling flips the answer.** Three customers are described by (annual income, monthly transactions): $P = (500000,\ 92)$, $Q = (500400,\ 10)$, $R = (480000,\ 90)$. (a) Using raw squared Euclidean distance, which of Q or R is closer to P? (b) Suppose the standard deviations of the two features across the whole dataset are $\sigma_{\text{income}} = 100000$ and $\sigma_{\text{transactions}} = 40$. Recompute after dividing each coordinate difference by its σ. Which customer is closer now, and which answer would you trust for a customer-segmentation task? Why?

4. **Breaking the guarantee.** Suppose someone modifies k-means so that the *update step* uses the **median** of each cluster, but keeps the squared-distance cost J. Consider the single cluster $\{1, 2, 10\}$. Compute J when the centre is its median and when it is its mean. Then explain precisely which half of the two-step convergence argument fails, and why the algorithm could in principle stop improving (or even get worse) while a better arrangement exists.

5. **Choosing c with evidence.** You run k-means on customer data for $c = 1, 2, \ldots, 6$ and obtain final costs $J = 600,\ 220,\ 90,\ 82,\ 78,\ 75$. (a) Why can J never be used on its own to pick the "best" c, and what value of c would you test the hardest here? Justify it by looking at the *size of each improvement*. (b) Your colleague argues that a medical-imaging task only needs to know whether a tissue type is present or absent. How does that change which c you start with, and why might $c = 3$ still be worth a try? (c) Finally, why can't any amount of J-analysis guarantee that the clusters correspond to the property the clinician cares about?